2 Create the transport object using the code in listing C.4. The `ElasticsearchTransport` object is constructed with the previously instantiated `restClient` instance and a JSON mapper (we use the Jackson JSON mapper in this instance).

Listing C.4 Construct Transport object

```
JacksonJsonpMapper jsonMapper = new JacksonJsonpMapper();

ElasticsearchTransport elasticsearchTransport =
   new RestClientTransport(restClient, jsonMapper);
```

Creates a new JacksonJsonpMapper object

Creates the transport object

We pass the `restClient` and `jsonMapper`, which were instantiated before creating the transport object.

1 Bring `ElasticsearchClient` to life with the following code.

Listing C.5 Construct the client

```
ElasticsearchClient elasticsearchClient =
       new ElasticsearchClient(elasticsearchTransport);
```

All `ElasticsearchClient` needs is the transport object created a moment ago. That's pretty much it—we have a client, and it's time to use it to interact with Elasticsearch.

C.5 Namespace clients

Earlier, we mentioned that the Java API client follows a pattern of requests, responses, and "feature" clients. Elasticsearch has a concept of package names and namespaces for each feature it exposes—for example, cluster-related APIs are available in the `*.cluster` package, indexing operations-related APIs in `*.index`, and so on. The Java API client follows the same pattern: a "client" is provided for every feature, as shown in figure C.1.

For example, all index-related classes such as requests, responses, and an indexing client exist under the `co.elastic.clients.elasticsearch.indices` package. The client for this namespace is `ElasticsearchIndicesClient` and is obtained from the main `ElasticsearchClient` (we see it working in a moment). All index operations are expected to be performed by this `ElasticsearchIndicesClient`. Similarly, all the other features follow the same pattern—a "folder" per feature (namespace), with each feature having a client named `ElasticsearchFEATUREClient`.

It may be overwhelming to see the number of classes, but the following examples of using the Java API client to fetch a feature/namespace client to support the feature clarify that the rest of the client specification follows the same approach. Let's see how to create an index.